

Days 6-10 — NumPy, Pandas, and statistics: complete study guide

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 2 foundations — definitions, real-world examples, implementations, and an interview-style quiz

Same format as the Days 1-5 and Week 3 guides: definition, real-world analogy, implementation, then a quiz. If any day below doesn't match your actual pdfs0X.py content, flag it and I'll correct that section.

Day 6 — NumPy arrays and indexing

Definition

Definition — NumPy array (ndarray): a fixed-type, fixed-size, multi-dimensional grid of numbers, stored contiguously in memory. Unlike a Python list, all elements share the same data type, which is what makes array operations dramatically faster than looping over a list.

Definition — Vectorized operation: an operation applied to an entire array at once (e.g. `array * 2`) instead of looping element by element, executed internally in fast compiled C code rather than the Python interpreter.

Real-world example

Real-world example: Converting every passenger's fare from dollars to a different currency by multiplying an entire fare column by an exchange rate in one line is vectorization. Doing the same thing with a Python for-loop over 10,000 rows would be noticeably slower and is exactly the habit NumPy (and pandas, built on top of it) trains you out of.

Implementation

```
import numpy as np

fares = np.array([7.25, 71.28, 8.05, 53.10])

fares_eur = fares * 0.92          # vectorized — no loop needed
print(fares_eur)

print(fares[0])                   # single element
print(fares[1:3])                 # slice
print(fares[fares > 20])          # boolean/fancy indexing

matrix = np.array([[1, 2], [3, 4]])
print(matrix.shape)              # (2, 2)
```

- `fares * 0.92` — multiplies every element by 0.92 in one operation, unlike a plain Python list which would need `{C}[f * 0.92 for f in fares]{E}`

- `fares[fares > 20]` — boolean indexing: `fares > 20` produces an array of True/False values the same length as `fares`, and indexing with it keeps only the elements where the condition was True. This exact mechanism is what powers pandas filtering like `df[df["Fare"] > 20]`
- `matrix.shape` — returns dimensions as a tuple; getting shape mismatches right is one of the most common sources of NumPy errors

Day 7 — NumPy aggregation, reshaping, and broadcasting

Definition

Definition — Aggregation: reducing an array to a summary statistic — sum, mean, min, max, std — either across the whole array or along a specific axis (rows vs columns).

Definition — Broadcasting: NumPy's rule set for performing operations between arrays of different shapes by 'stretching' the smaller array's dimensions to match the larger one, without actually copying data.

Real-world example

Real-world example: Computing the average fare per passenger class from a 2D grid of fares by class is axis-based aggregation (mean along one axis, not the whole array). Broadcasting is what lets you subtract a single average fare value from every row of a fare column in one line, without manually repeating that average into an array the same length first.

Implementation

```
import numpy as np

ages = np.array([22, 38, 26, 35, 28])
print(ages.mean(), ages.std(), ages.min(), ages.max())

grid = np.array([[22, 71.28], [38, 53.10], [26, 8.05]]) # [age, fare] per passenger
print(grid.mean(axis=0)) # column-wise mean: [mean_age, mean_fare]
print(grid.mean(axis=1)) # row-wise mean per passenger

centered = ages - ages.mean() # broadcasting: scalar subtracted from every element
```

- `axis=0` aggregates down each column (one result per column); `axis=1` aggregates across each row (one result per row) — mixing these up is one of the most common NumPy/pandas bugs when computing per-column vs per-row statistics
- `ages - ages.mean()` — `ages.mean()` is a single number, but NumPy broadcasts it against every element of the array automatically, no manual repeat needed

Day 8 — Pandas Series and DataFrames

Definition

Definition — Series: a one-dimensional labeled array, essentially a single column with an index attached.

Definition — DataFrame: a two-dimensional labeled table — a collection of Series sharing the same index, which is the core data structure every script from Day 10 onward is built around.

Real-world example

Real-world example: Reading a Titanic CSV into a DataFrame with `pd.read_csv()` is the single most-used line in this entire roadmap. Selecting one column (`df["Age"]`) gives back a Series; selecting several columns (`df[["Age", "Fare"]]`) gives back a DataFrame — the same single-bracket vs double-bracket distinction that trips up almost every beginner at least once.

Implementation

```
import pandas as pd

df = pd.read_csv("titanic.csv")

print(df.head())          # first 5 rows
print(df.shape)           # (rows, columns)
print(df.info())          # column types + non-null counts

ages = df["Age"]           # Series
subset = df[["Age", "Fare"]] # DataFrame

row5 = df.iloc[5]          # position-based row access
row_by_label = df.loc[5]   # label-based row access (same here since index is default)
```

- `df.head()` and `df.info()` are almost always the first two calls made on any new dataset — a quick shape/preview check before doing anything else
- `.iloc[]` is purely position-based (like list indexing); `.loc[]` is label-based, using the actual index values — they behave identically only when the index happens to be the default 0, 1, 2... range

Day 9 — Filtering, groupby, merging, and missing data

Definition

Definition — Boolean filtering: selecting rows of a DataFrame that satisfy a condition, using a boolean mask like `df[df["Age"] > 30]`.

Definition — groupby: splits a DataFrame into groups based on one or more column values, applies an aggregation to each group independently, then combines the results back into one table ("split-apply-combine").

Definition — Missing data (NaN): pandas' representation of an absent value. Handled with `isna()` to detect it, and `fillna()/dropna()` to fix it — exactly the `fillna(df["Age"].median())` pattern used in every cleaning function from Day 10 onward.

Real-world example

Real-world example: "What's the average fare paid by passengers in each class?" is a one-line `groupby`: `df.groupby("Pclass")["Fare"].mean()`. This is the pandas equivalent of the GROUP BY + AVG() SQL query practiced in Day 11 — the same underlying idea, two different tools.

Implementation

```
import pandas as pd

older = df[df["Age"] > 30]
older_men = df[(df["Age"] > 30) & (df["Sex"] == "male")]

avg_fare_by_class = df.groupby("Pclass")["Fare"].mean()

left = pd.DataFrame({"Pclass": [1, 2, 3], "Label": ["First", "Second", "Third"]})
merged = df.merge(left, on="Pclass", how="left")

print(df["Age"].isna().sum())          # count missing values
df["Age"] = df["Age"].fillna(df["Age"].median())
```

- combining conditions needs `&` (and) or `|` (or), not Python's `and/or`, and each condition must be wrapped in parentheses — `(df["Age"] > 30) & (df["Sex"] == "male")` is required syntax, not a style choice
- `df.merge(left, on="Pclass", how="left")` — `how="left"` keeps every row from `df` even if there's no match in `left`, the pandas equivalent of a SQL LEFT JOIN
- `fillna(df["Age"].median())` — this exact line appears in every single cleaning function from Day 10 through Day 19, always using median rather than mean because Age is right-skewed

Day 10 — Statistics fundamentals

Definition

Definition — Descriptive statistics: summary numbers that describe a dataset's center (mean, median, mode), spread (variance, standard deviation, IQR), and shape (skew), without making any claims beyond the data itself.

Definition — Hypothesis testing (t-test, chi-square): a formal procedure for deciding whether an observed difference between groups (like survival rates between sexes) is likely real or could plausibly be explained by random chance alone, expressed through a p-value.

Definition — Correlation vs causation: correlation measures how strongly two variables move together (a number between -1 and 1); it never by itself proves that one variable causes changes in the other.

Real-world example

Real-world example: "Passengers in first class survived at a higher rate than third class — is that a real pattern or could it be random noise?" is exactly what a chi-square test answers. Ice cream sales and drowning deaths both rise in summer and correlate strongly, but neither causes the other — both are driven by a third factor, heat — the textbook example of why correlation isn't causation.

Implementation

```
import pandas as pd
from scipy import stats

print(df["Age"].mean(), df["Age"].median(), df["Age"].std())

q1, q3 = df["Fare"].quantile([0.25, 0.75])
iqr = q3 - q1 # interquartile range, used to define outlier fences

male_ages = df[df["Sex"] == "male"]["Age"]
female_ages = df[df["Sex"] == "female"]["Age"]
t_stat, p_value = stats.ttest_ind(male_ages, female_ages, equal_var=False)

contingency = pd.crosstab(df["Sex"], df["Survived"])
chi2, p_value_chi, dof, expected = stats.chi2_contingency(contingency)
```

- `equal_var=False` — runs Welch's t-test instead of the standard t-test, which doesn't assume both groups have equal variance; safer as a default unless you've specifically checked that assumption holds
- `p_value < 0.05` is the conventional (not universal) cutoff for calling a result "statistically significant" — it means there's less than a 5% chance of seeing a difference this large if there were actually no real difference
- `pd.crosstab()` builds the contingency table `chi2_contingency` needs — a grid of counts for every combination of two categorical variables

Interview prep quiz — technical

Questions about NumPy/Pandas syntax and statistical implementation details.

Q: What's the practical difference between a NumPy array and a Python list for numerical work?

A: A NumPy array requires every element to share the same data type and is stored contiguously in memory, enabling vectorized operations that run in compiled C code. A Python list can hold mixed types but operations on it require slower, interpreted loops.

Q: What does `axis=0` vs `axis=1` mean in a pandas or NumPy aggregation, and why is mixing them up a common bug?

A: `axis=0` aggregates down each column, producing one result per column; `axis=1` aggregates across each row, producing one result per row. Confusing them silently produces a result of the wrong shape or meaning rather than an error, which makes the bug easy to miss.

Q: Why does `df["Age"]` return a Series but `df[["Age"]]` return a DataFrame?

A: Single brackets with one column name select that column directly as a 1D Series. Double brackets pass a list of column names, and selecting by a list always returns a 2D DataFrame, even if the list has only one element.

Q: How do you combine two filter conditions on a DataFrame, and why can't you use Python's normal and/or?

A: Use `&` for and, `|` for or, with each condition wrapped in parentheses: `df[(cond1) & (cond2)]`. Python's and/or expect single boolean values, but each condition here produces an entire array of booleans, which needs pandas' element-wise operators instead.

Q: What's the difference between `.loc[]` and `.iloc[]` in pandas?

A: `.iloc[]` selects by integer position, like list indexing. `.loc[]` selects by label, using the actual index values. They only behave identically when the DataFrame still has its default 0,1,2... integer index.

Q: What does `groupby("Pclass")["Fare"].mean()` actually do step by step?

A: It splits the DataFrame into separate groups, one per unique Pclass value, applies `.mean()` to the Fare column within each group independently, then combines the per-group results back into a single Series indexed by Pclass — the 'split-apply-combine' pattern.

Q: Why is `equal_var=False` used when running a t-test between two groups?

A: It switches to Welch's t-test, which doesn't assume the two groups have equal variance. Since that assumption often doesn't hold in real data, Welch's version is the safer default unless variance equality has actually been checked.

Q: What does `pd.crosstab()` produce, and why is it needed before `chi2_contingency()`?

A: It builds a contingency table — a grid of counts for every combination of two categorical variables. `chi2_contingency()` requires that count grid as its input; it can't work directly on the raw columns.

Interview prep quiz — theoretical / conceptual

Questions about why these tools and statistical ideas matter in real analysis work.

Q: Why does pandas (built on NumPy) dominate real-world data work instead of just using plain Python lists and dictionaries?

A: Vectorized operations are dramatically faster at scale, the labeled index makes joining and aligning data reliable, and built-in handling for missing data, grouping, and file I/O covers the vast majority of real data-cleaning needs without hand-written loops.

Q: Why should you always check `df.isna().sum()` before doing any analysis on a new dataset?

A: Missing values silently break downstream calculations — a mean computed while missing values are still masked as strings or nulls can produce wrong or NaN results without any error being raised. Knowing exactly which columns and how many rows are affected shapes every cleaning decision that follows.

Q: Why is 'correlation is not causation' such a commonly repeated warning in data science, and what does it actually protect against?

A: Two variables can move together because one causes the other, because both are driven by a hidden third factor, or by pure coincidence. Treating a strong correlation as proof of causation leads to wrong conclusions and, in a business setting, wrong decisions — like assuming a marketing campaign caused a sales increase that was actually seasonal.

Q: Why does the choice between mean and median matter when summarizing a skewed variable like Fare or Age?

A: The mean is pulled toward extreme values (a few very expensive first-class fares can drag the average fare up), while the median represents the actual middle of the distribution regardless of outliers. Reporting only the mean on a skewed variable can misrepresent what's 'typical'.